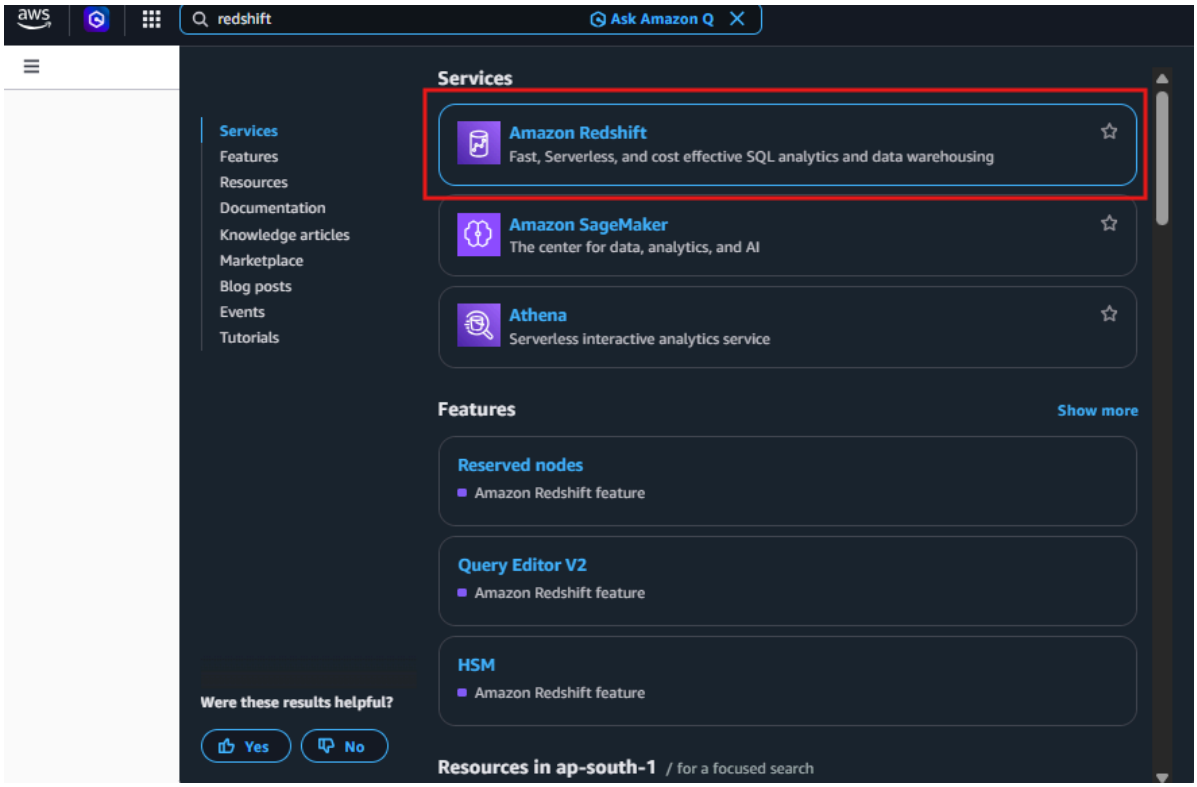
Lab: Ingest Extracted Data into Amazon Redshift Raw Layer Using AWS Glue

Objective

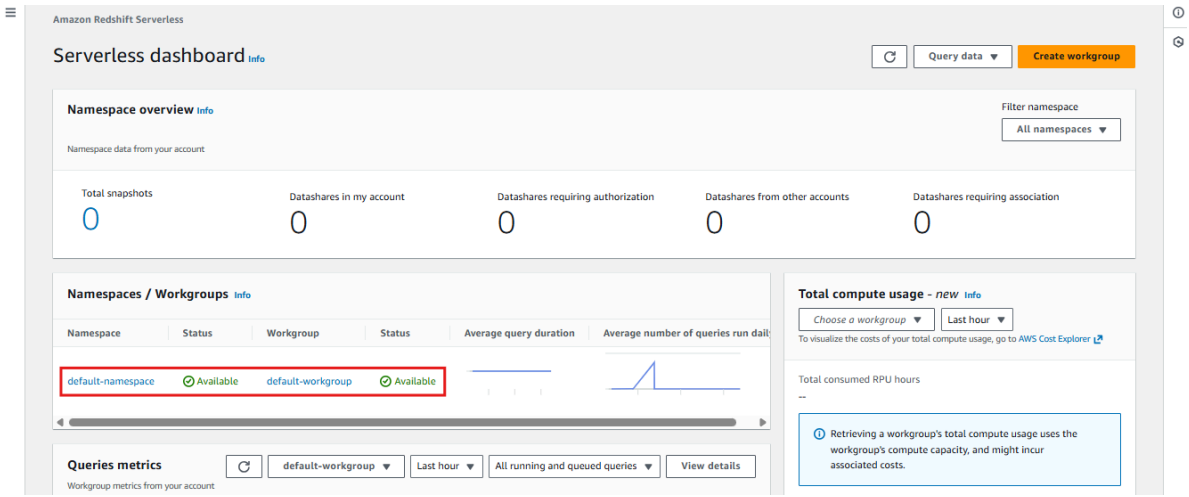
Create Redshift raw and process schemas, design tables, and deploy an AWS Glue Python Shell job to ingest extracted S3 data into Redshift using COPY and MERGE operations.

Step 1: Create Database and Schemas in Redshift

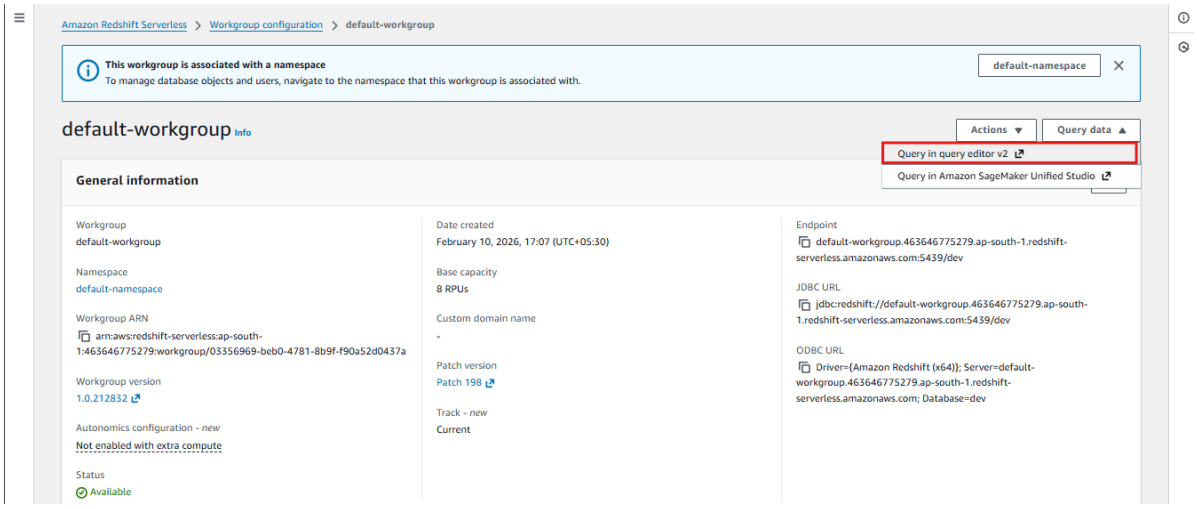
1. Open AWS Console and search for **Redshift**.



2. Select your existing **Serverless workgroup**.

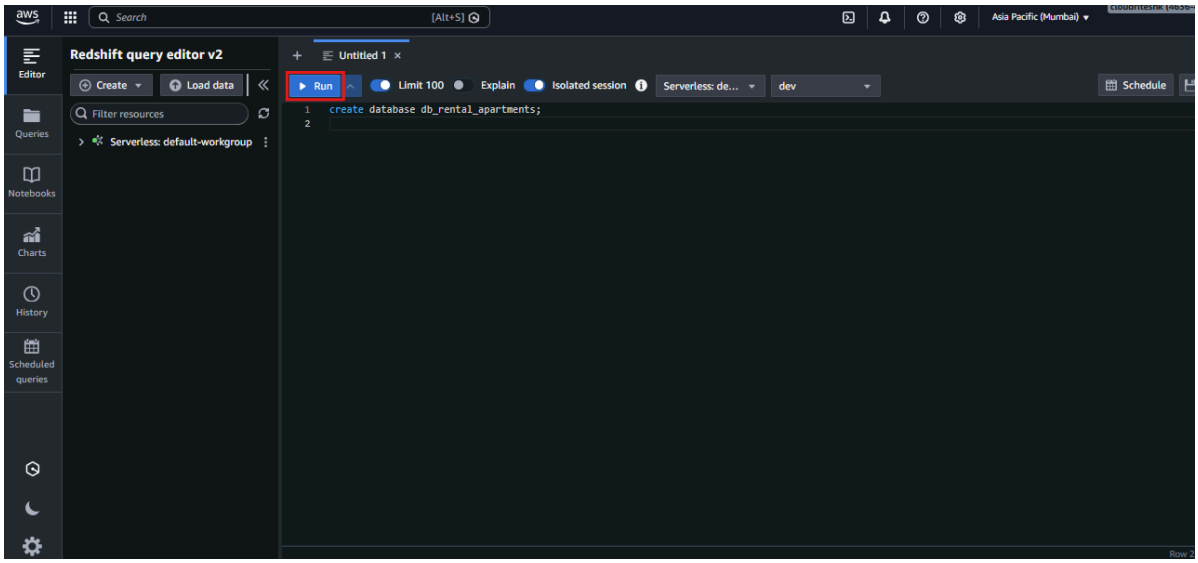


3. Click **Query Data**.



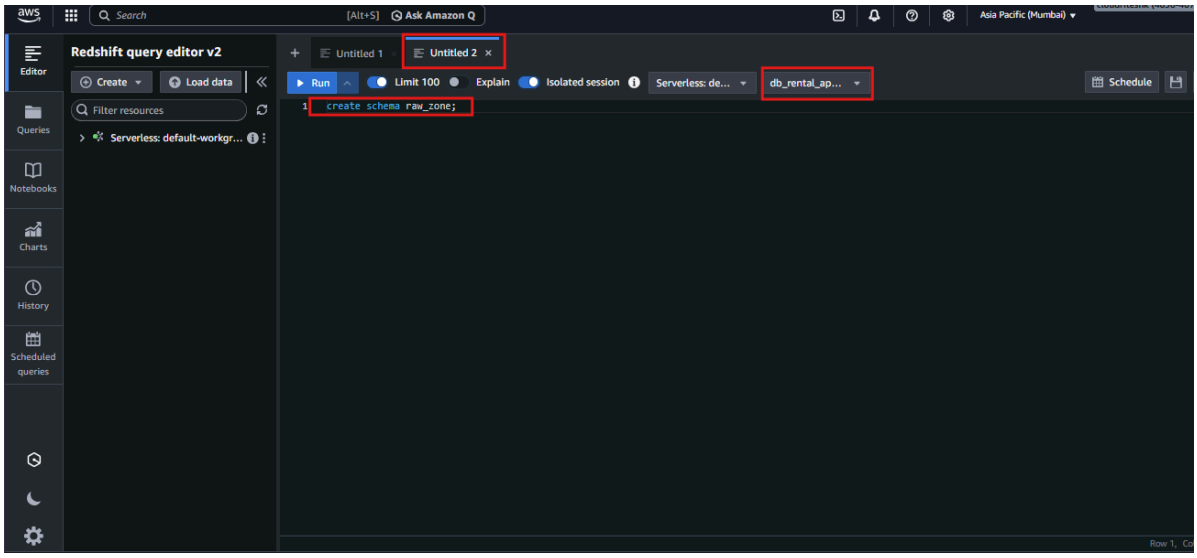
4. Execute the following command:

```
CREATE DATABASE db_rental_apartments;
```

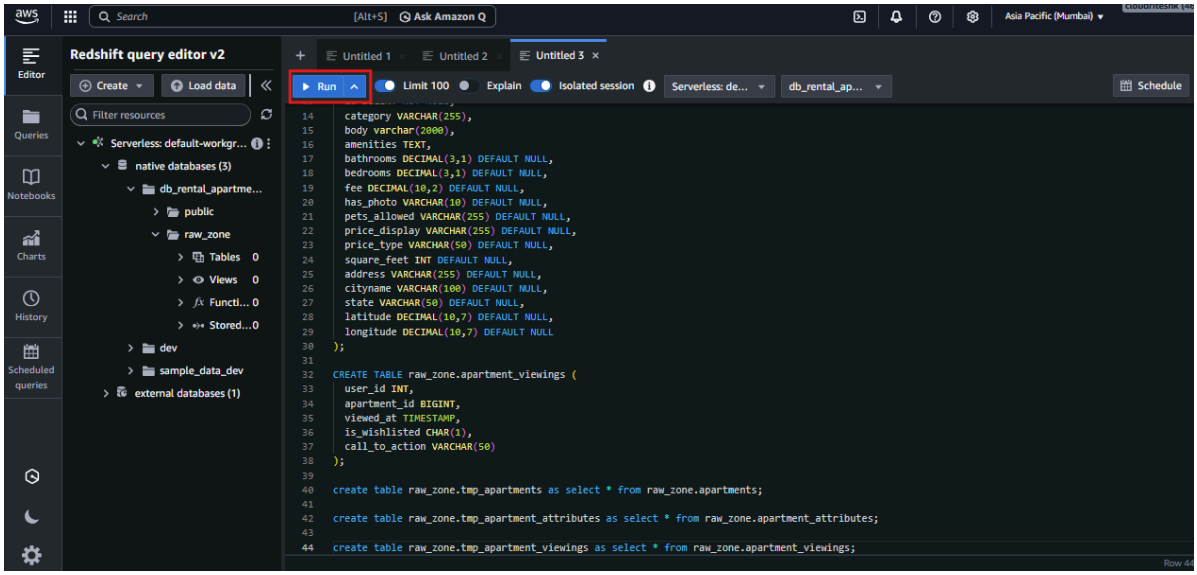


5. Open a new query editor tab.
6. Select database: `db_rental_apartments`.
7. Create raw schema:

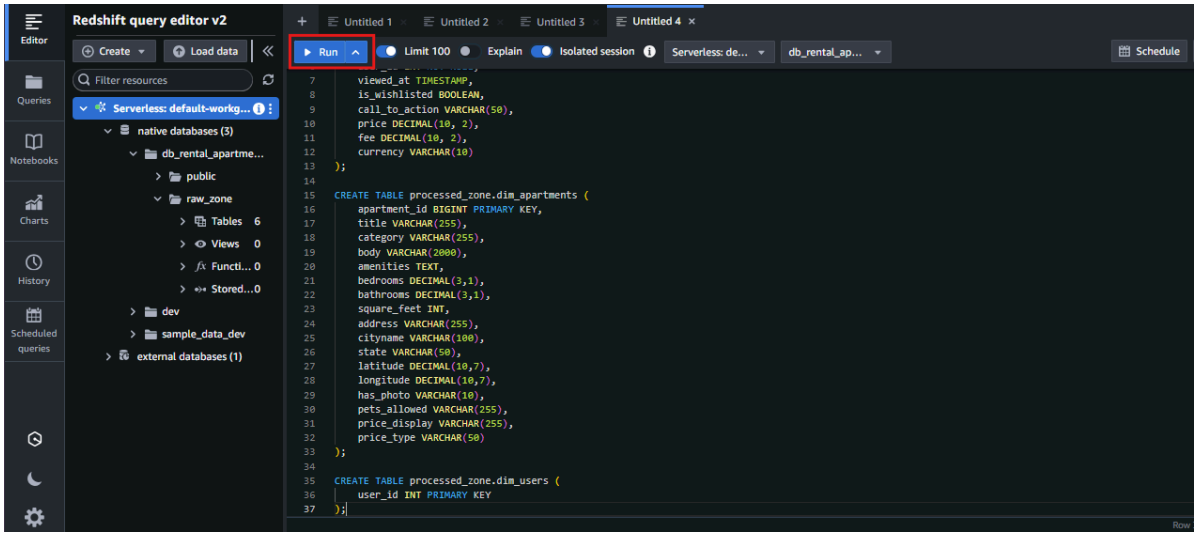
```
CREATE SCHEMA raw_zone;
```



8. Open a new query editor tab and execute the remaining commands in one go.

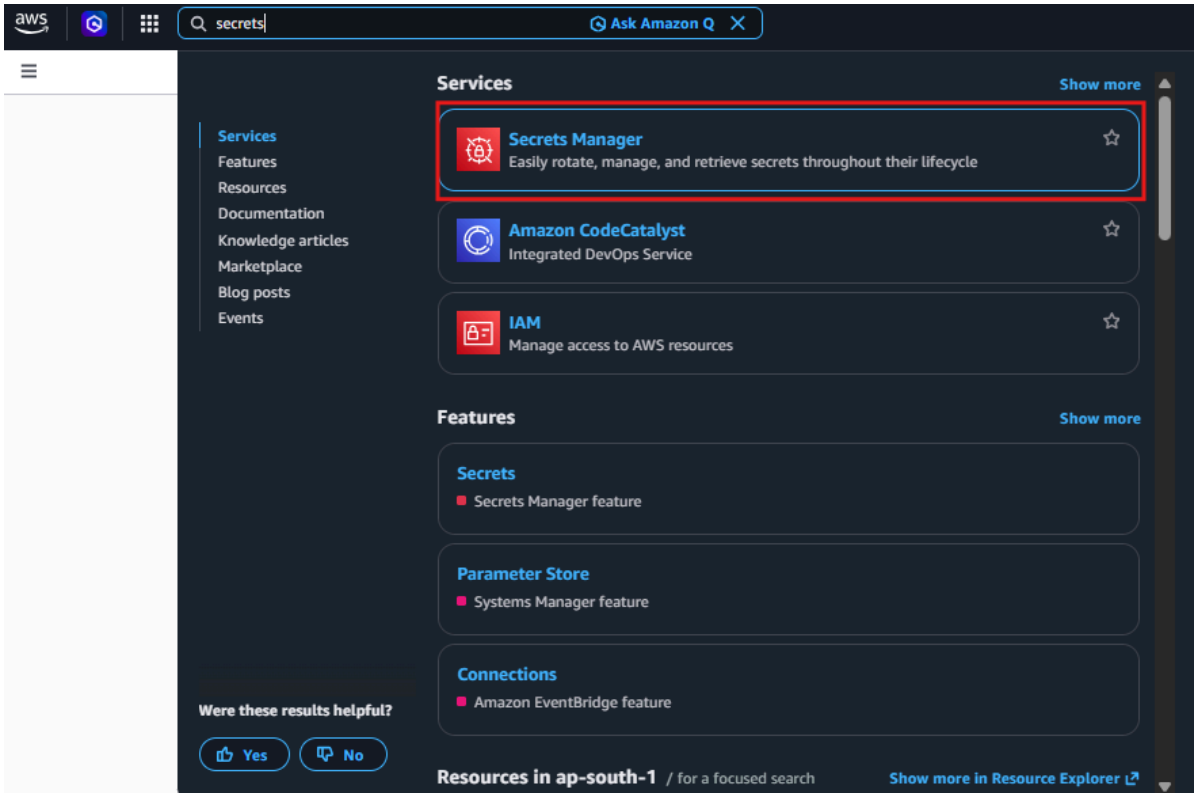


9. Similarly Create process schema:

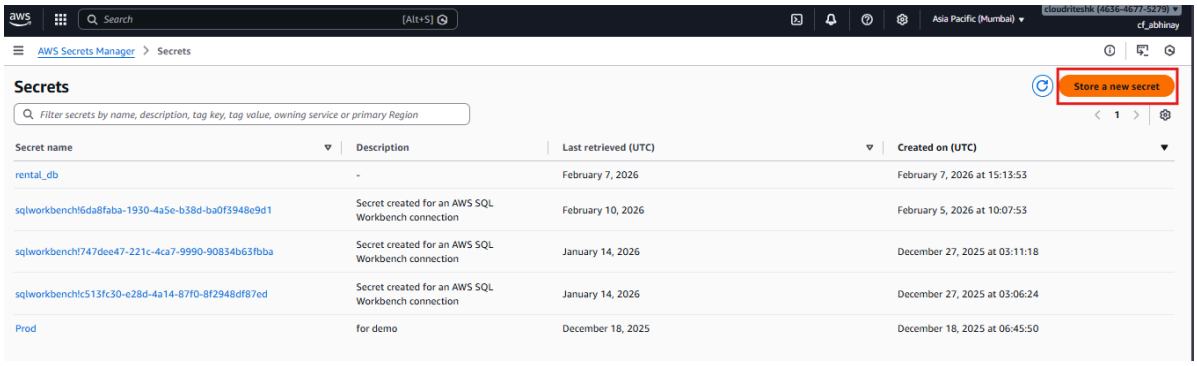


Step 2: Create Secret for Redshift Credentials

1. Navigate to **AWS Secrets Manager**.

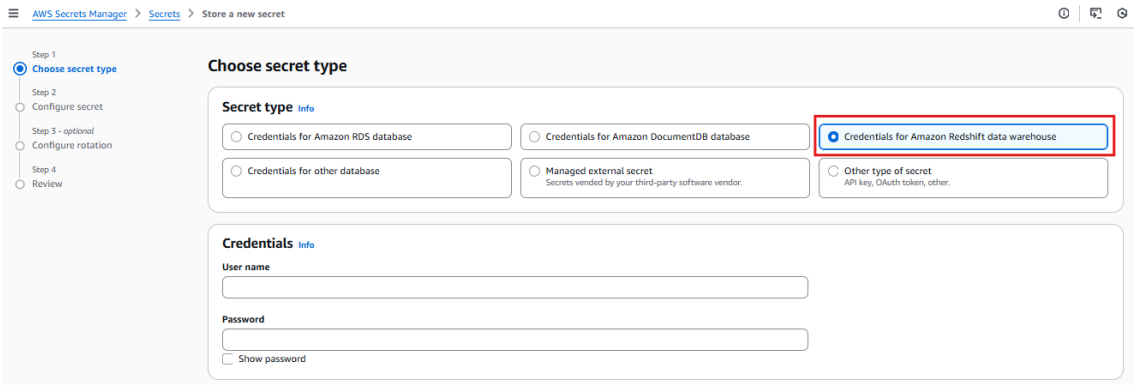


2. Click **Store a new secret**.

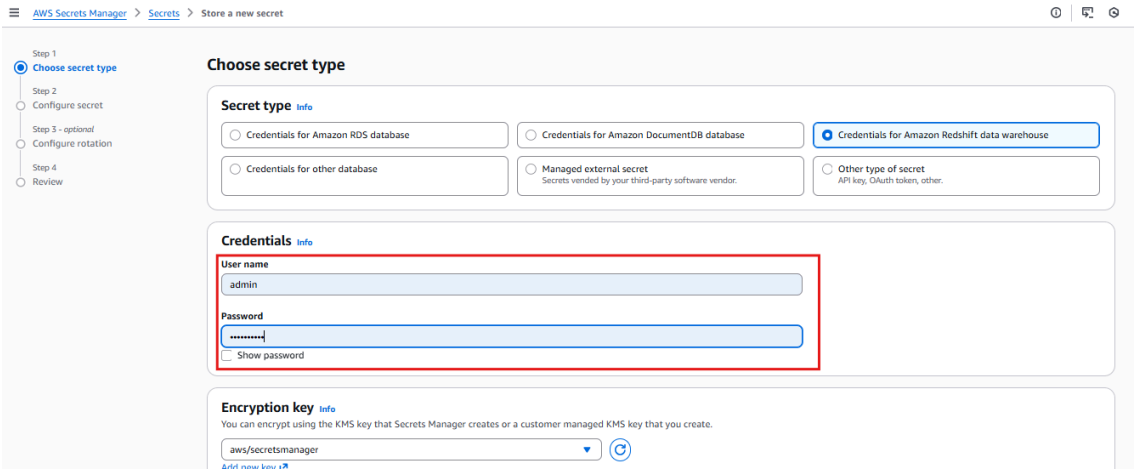


3. Select:

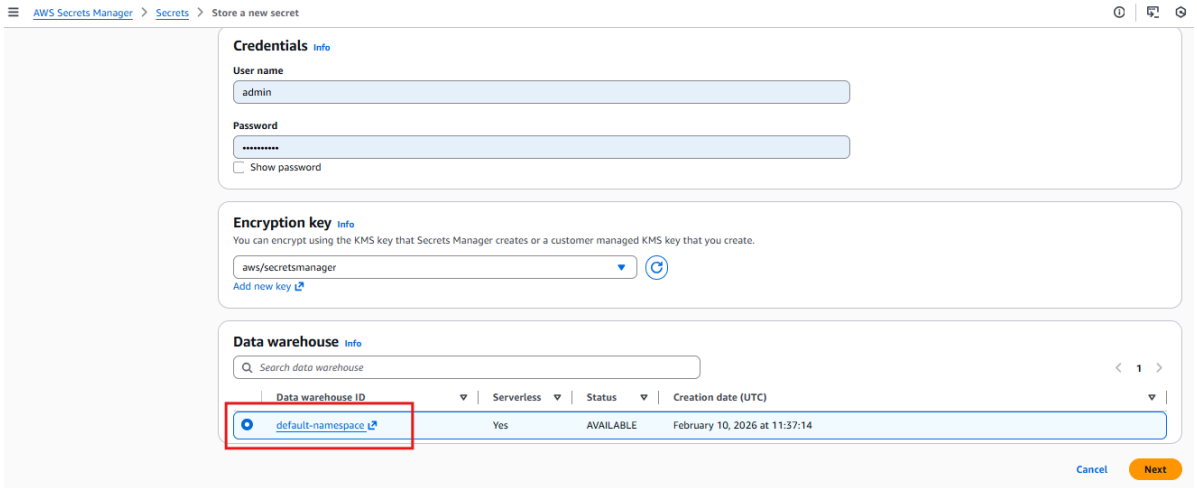
   - Credentials for Redshift data warehouse.



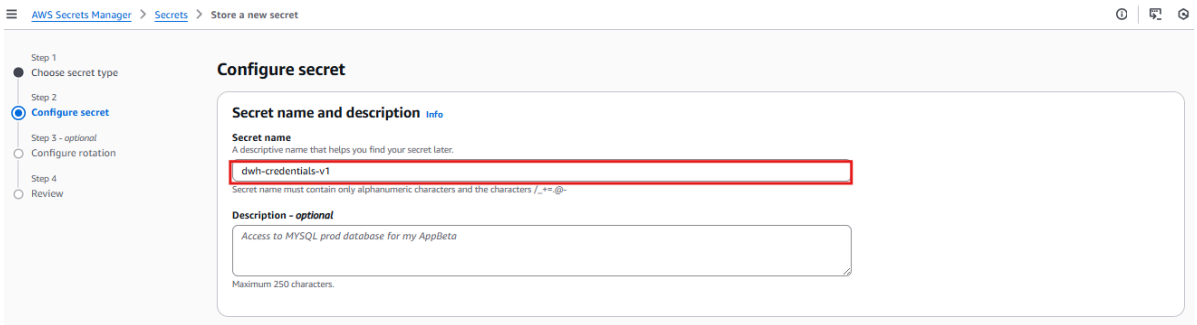
4. Enter:
    - Username
    - Password



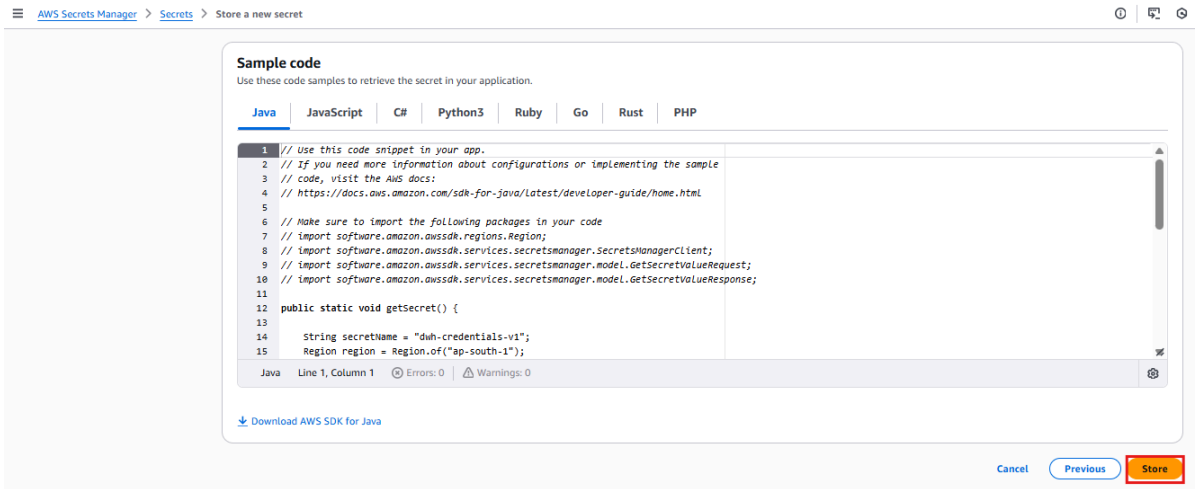
5. Select your Redshift namespace.



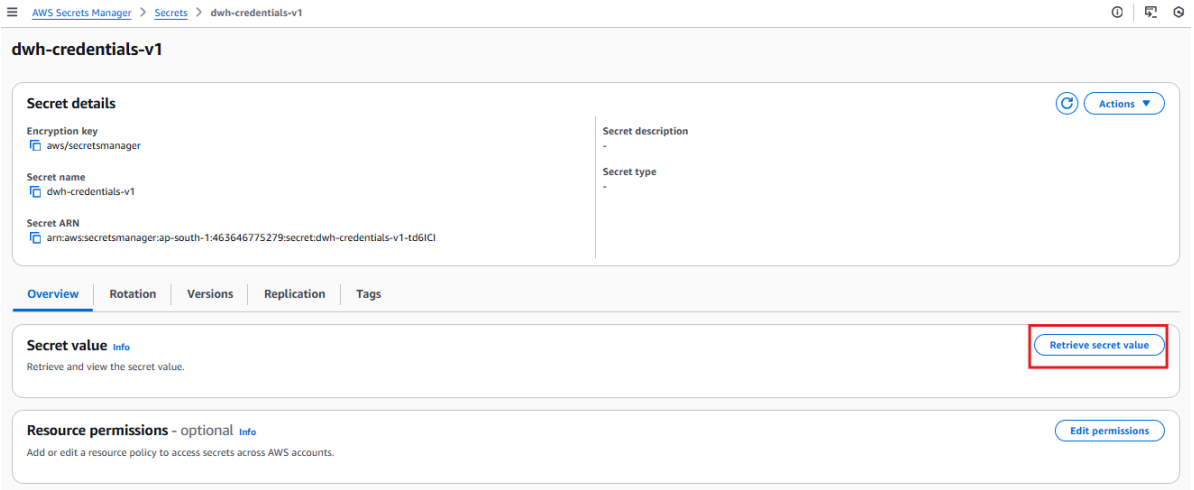
6. Secret name → `DWCredentials` (or similar).

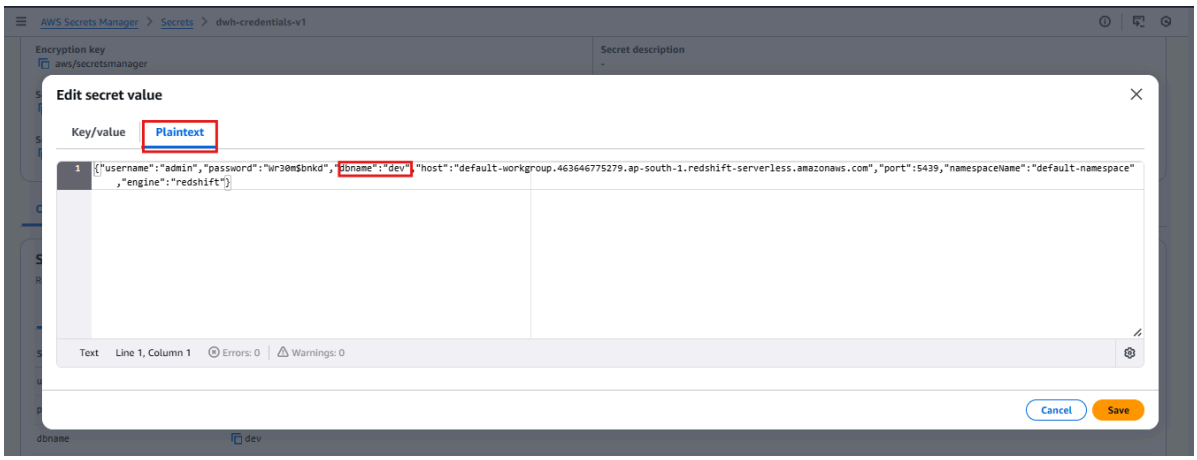


7. Click **Store**.

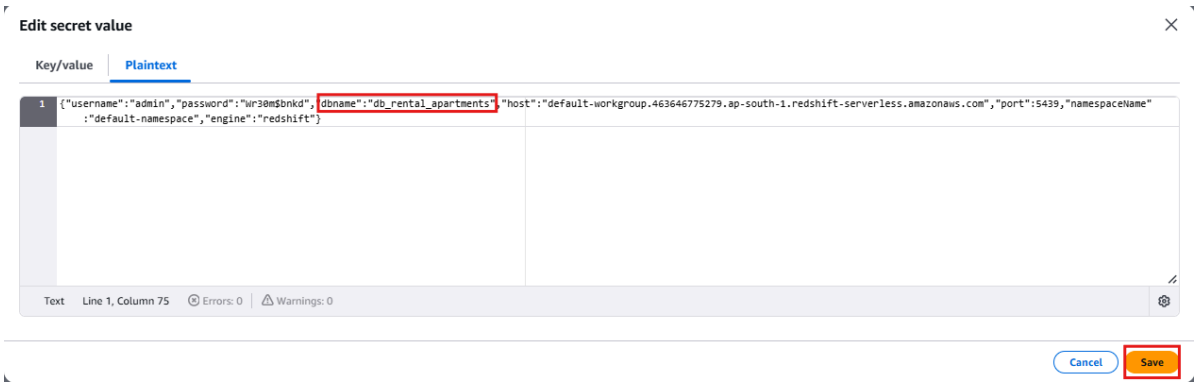


8. Go to Retrieve secret value ⟶ Edit secret:
   - Change `"dbname"` from `dev` to `db_rental_apartments`.



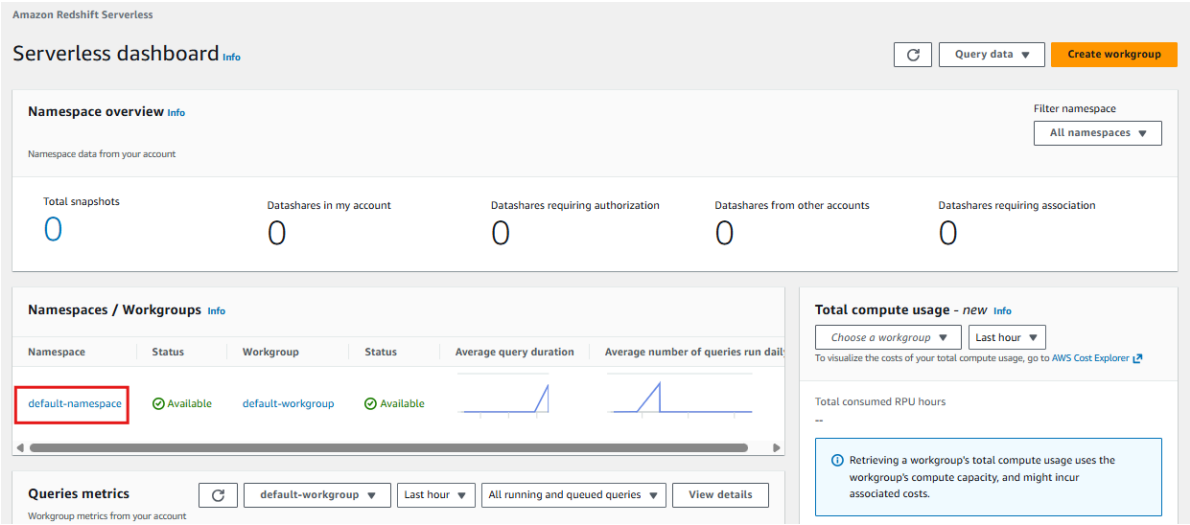


9. Save changes.

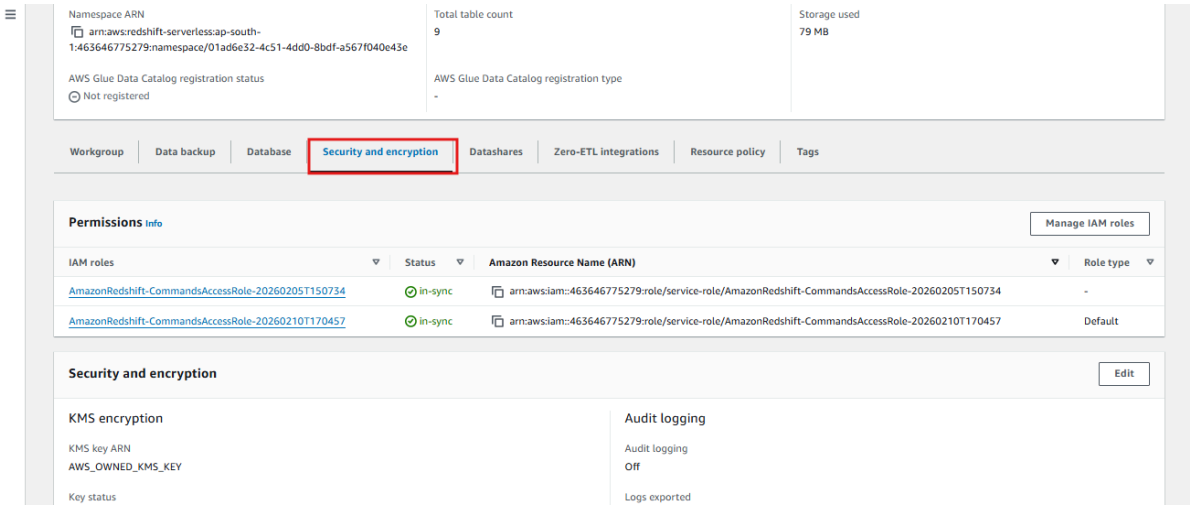


Step 3: Retrieve Redshift IAM Role ARN

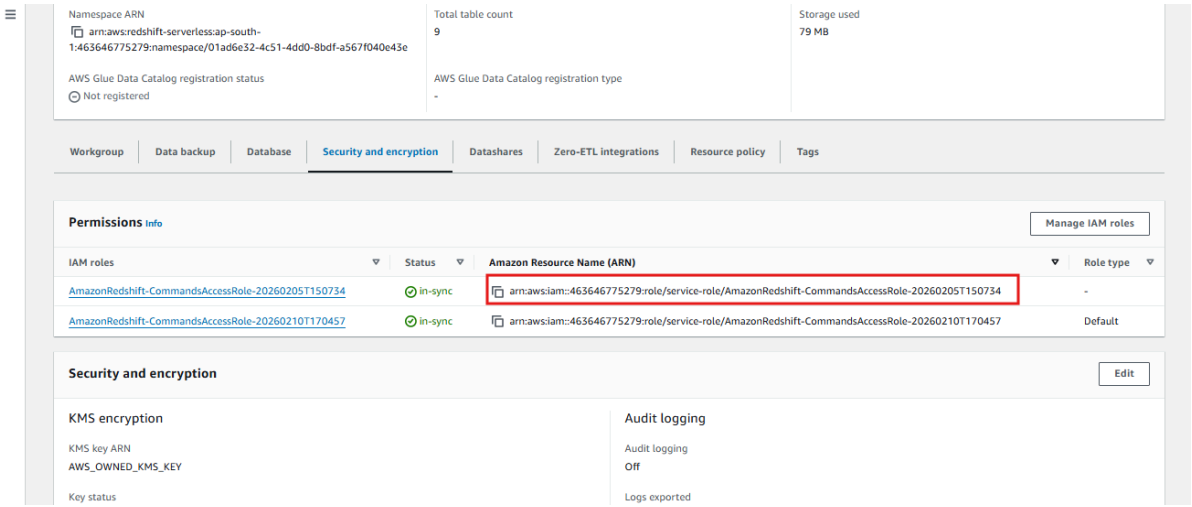
1. Open Redshift → Namespace.



2. Go to **Security and Encryption**.



3. Copy IAM role ARN attached to Redshift.



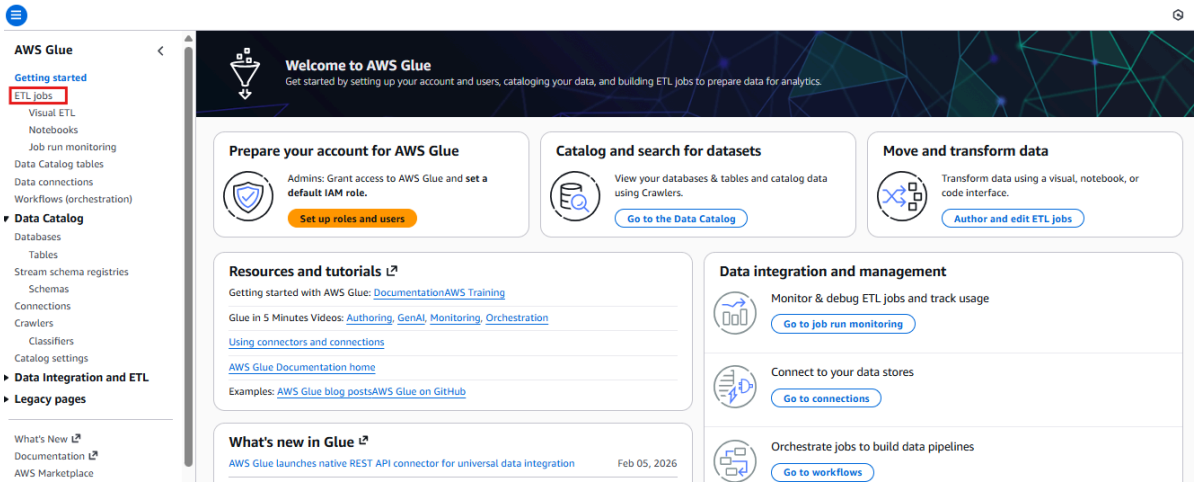
4. Paste ARN value into script variable:

```
redshift_iam_arn = "your-iam-role-arn"
```

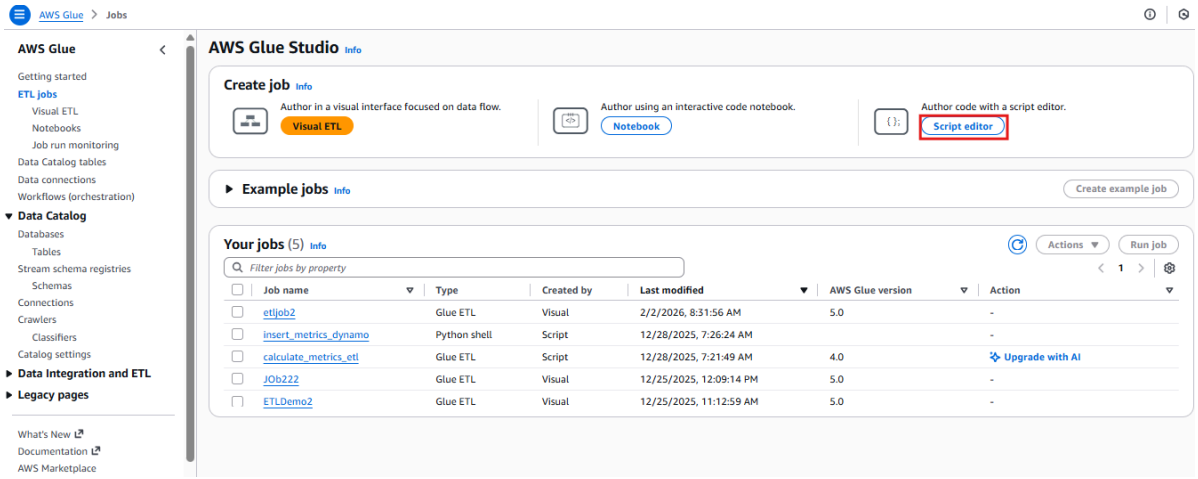
5. Also verify:
   - Correct S3 bucket name.
   - Correct AWS region.

Step 4: Create Glue Python Shell Job for Redshift Ingestion

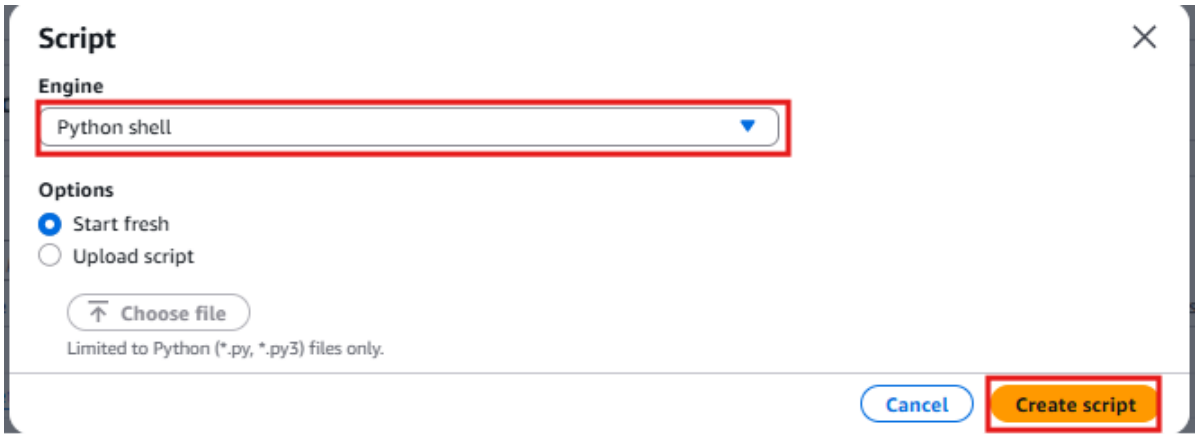
1. Navigate to **AWS Glue** → **ETL Jobs**.



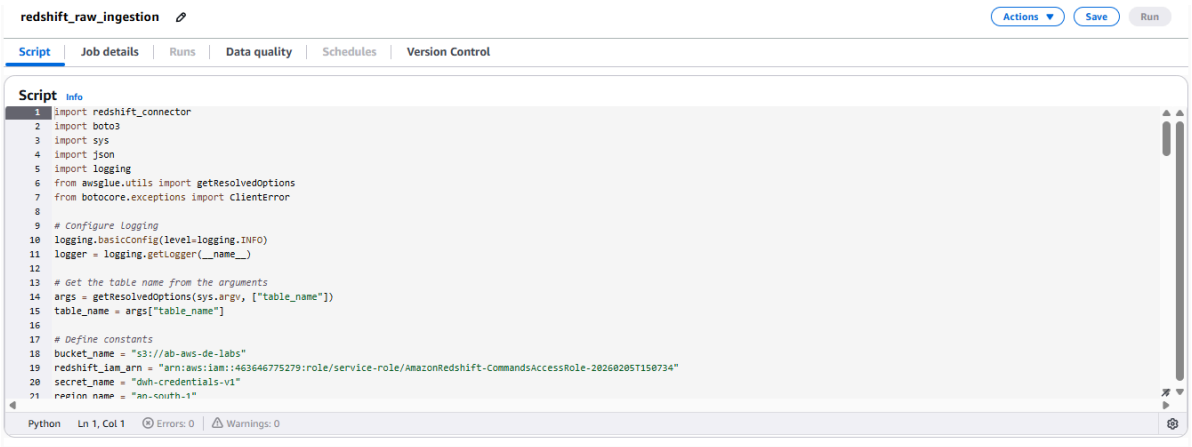
2. Click **Script Editor**.



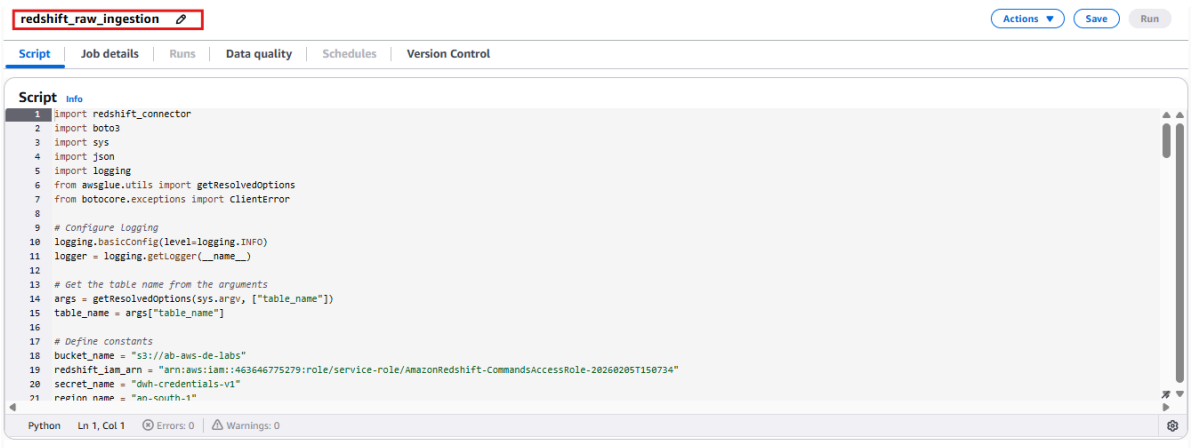
3. Select **Python Shell**.



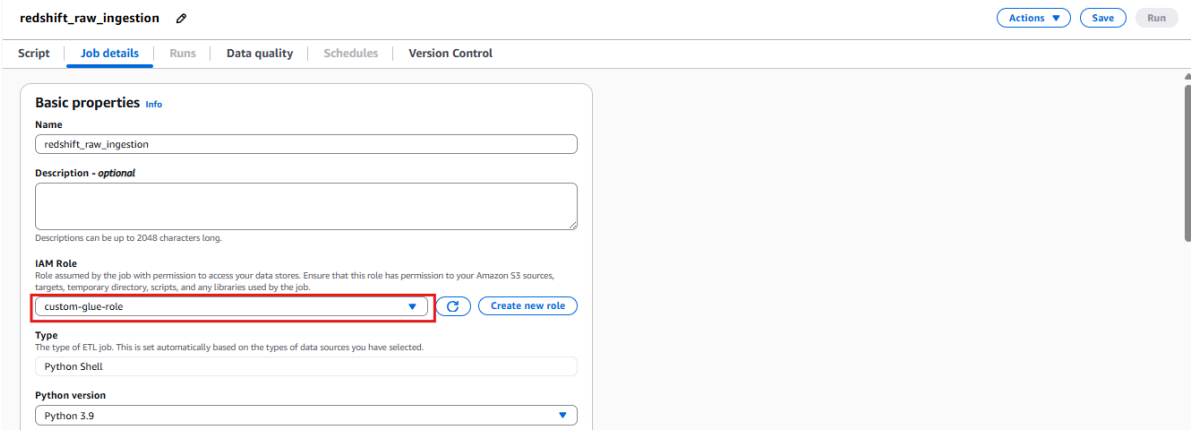
4. Paste contents of `redshift_raw_ingestion.py`.

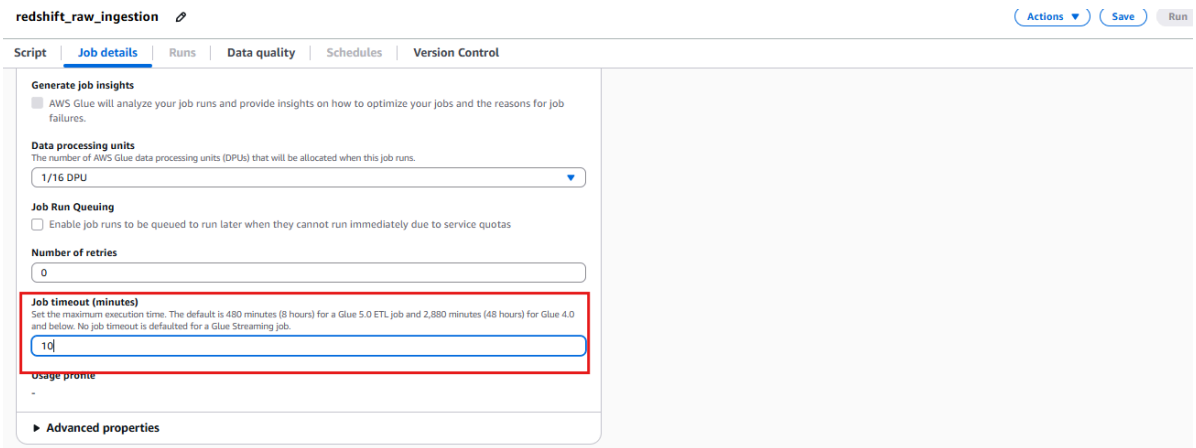


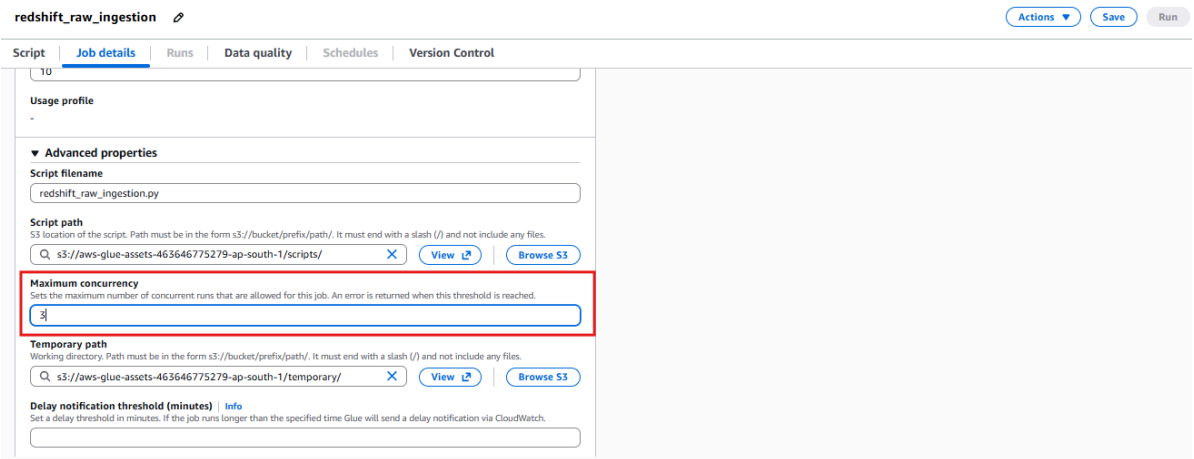
5. Job name → `RedshiftRawIngestion`.



6. Under Job Details:
   - Select IAM role (must have S3, Redshift, SecretsManager access).
   - Timeout → 10 minutes.
   - Max concurrency → 3.

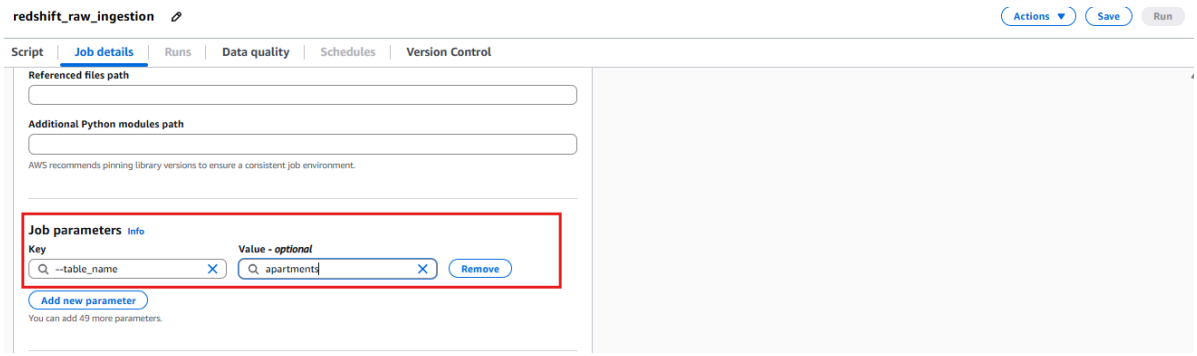




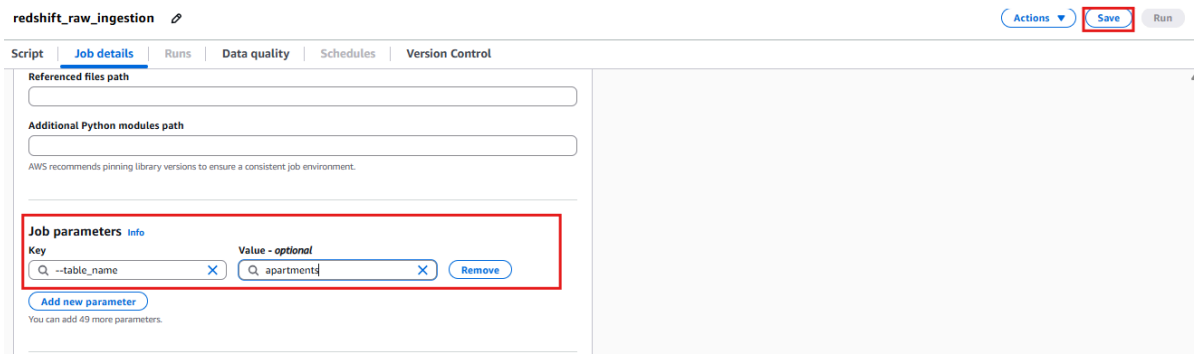


7. Add Job Parameter:

   - `-table_name` → apartments.



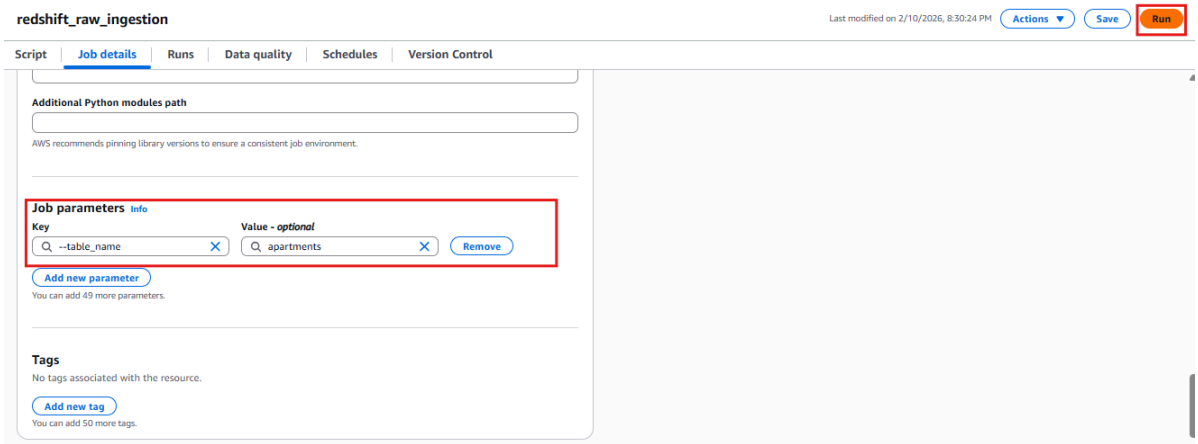
8. Save the job.



Step 5: Execute Glue Job for Each Table

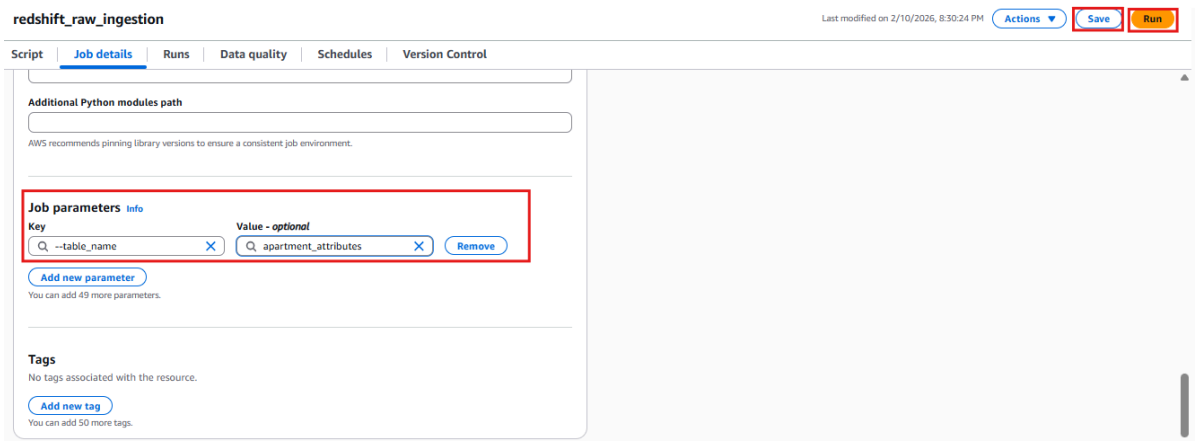
1. Run job with:

   - `apartments`



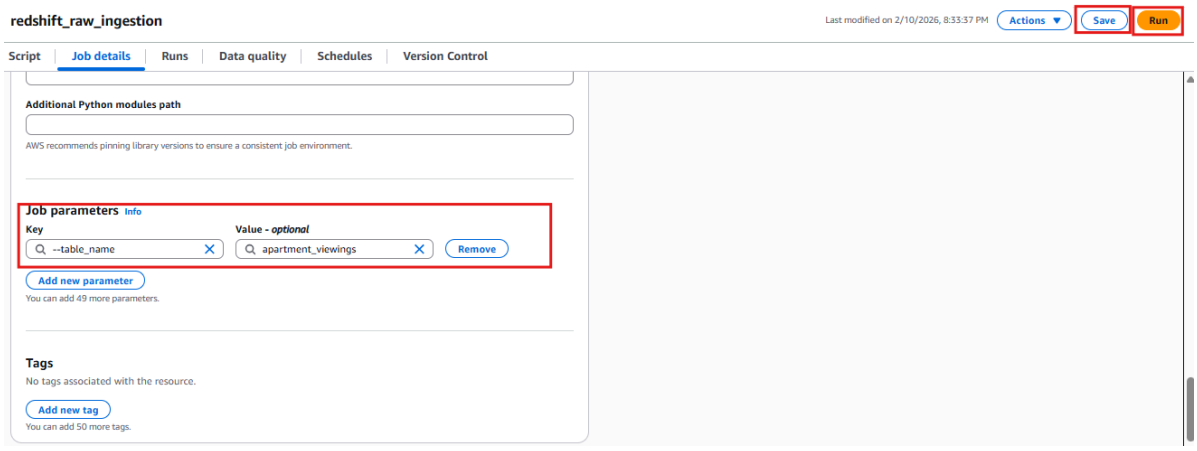
2. Modify parameter and run for:

   - `apartment_attributes`



3. Modify parameter and run for:

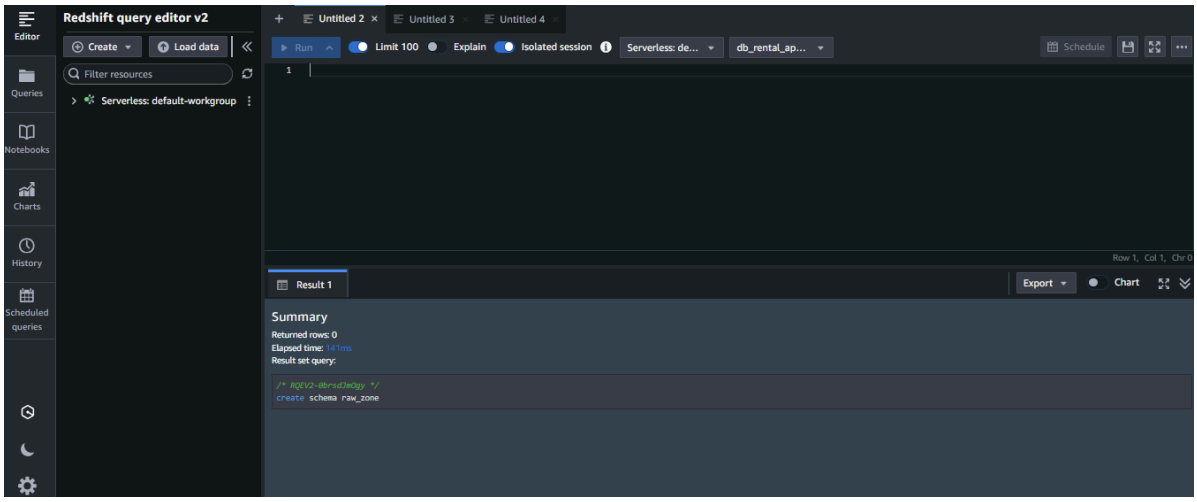
   - `apartment_viewings`



4. Monitor job runs until status shows **Succeeded**.

Step 6: Verify Data in Redshift

1. Open Redshift → Query Editor.



2. Select database `db_rental_apartments`.
3. Verify data:

```
SELECT * FROM raw_zone.apartments LIMIT 10;
SELECT * FROM raw_zone.apartment_attributes LIMIT 10;
SELECT * FROM raw_zone.apartment_viewings LIMIT 10;
```

4. Verify temp tables are empty:

```
SELECT * FROM raw_zone.tmp_apartments;
```

Temp tables should return zero rows after truncation.

Outcome

Raw layer database and schemas were successfully created in Redshift.

Glue Python Shell job ingested data from S3 into Redshift using COPY and MERGE operations.

Temporary staging tables were cleared after ingestion.

Data is now available in the raw layer and ready for transformation into the process layer.